

System Design Case Study

Provenance & Auditability for an Agentic Investment Platform

Context

Soma Capital is building an internal investment platform powered by AI agents.

These agents autonomously assist the investment team by:

- **Sourcing deals** – scanning databases, news, and social signals
- **Enriching data** – pulling financials, founder history, and market context
- **Drafting outreach** – generating emails to founders and LPs
- **Updating systems** – syncing CRMs and internal tools
- **Generating investment memos** – summarizing diligence and market analysis

This creates a powerful system — but also introduces risk.

AI agents are **non-deterministic**. They can hallucinate, change behavior between runs, and depend on external APIs. At the same time, venture firms increasingly need strong guarantees around:

- **Data provenance**
- **Auditability**
- **Decision traceability**

LPs (limited partners), internal partners, and regulators must be able to understand:

Who produced a piece of information, from which inputs, and whether it has been modified.

Your task is to design the infrastructure that makes this possible.

The Problem

Design a **provenance and auditability system** for an AI-driven investment platform.

The system should provide **cryptographic or verifiable guarantees** about artifacts produced by both deterministic and non-deterministic workflows.

Artifacts may include:

- Investment memos
 - CRM updates
 - Founder profiles
 - Enriched datasets
 - Drafted emails
 - Due diligence notes
-

Requirements

Your system should address the following areas.

1. Authenticated Provenance

Every artifact must have a verifiable chain of custody.

The system should allow us to answer questions like:

- Which **agent or human** created this artifact?
- What **inputs or sources** were used?
- What **version of the agent** produced it?
- When was it created?
- Has it been **modified since creation**?

Design a system that ensures artifacts cannot be silently altered.

2. Durable Workflow Orchestration

Agent workflows are complex and failure-prone.

Failures may include:

- API timeouts
- partial execution
- retries
- inconsistent data
- LLM hallucinations

Design an orchestration system that:

- Ensures workflows complete reliably
- Supports retries and checkpointing
- Preserves provenance through retries
- Maintains traceability across multi-step workflows

Your design should address the boundary between:

- **deterministic steps** (database writes, API calls)
 - **non-deterministic steps** (LLM inference, agent reasoning)
-

3. Identity & Key Management

Both humans and agents should have cryptographic identities.

Your system should address:

- How agents obtain signing keys
- How identities are issued and revoked
- Key rotation strategies
- Delegation of authority
- Preventing impersonation or tampering

Describe the **trust model** for the system.

4. Audit Query Layer

Assume an LP asks the following question:

“Show me every action and data source that contributed to the decision to invest in Company X.”

Design the system that allows auditors to reconstruct this history.

Consider:

- Query interfaces
 - Underlying storage model
 - Graph traversal
 - Latency vs storage tradeoffs
 - Generating verifiable proofs for external reviewers
-

System Constraints

Assume the following scale:

- ~5,000 agent actions per day today
- ~50,000 actions per day within 2 years
- Artifact sizes range from **100 bytes to ~50 KB**
- Some workflows run in **seconds**
- Others span **hours or days**

Additional constraints:

- Multiple cloud services
 - Multiple external APIs
 - Internal and external users
 - Audit queries should complete within **minutes**
-

Deliverables

Please provide:

1. High-Level Architecture

Describe the overall system architecture, including:

- Major components
- Data flow
- Trust boundaries
- Storage layers

A diagram is encouraged but not required.

2. Deep Dive

Choose **one component** of the system and explore it in depth.

Examples:

- Provenance data structure
- Workflow orchestration engine
- Agent identity model
- Audit query system
- Handling non-deterministic AI steps

We are looking for **depth of thinking**, not breadth.

3. Tradeoffs

Discuss:

- What you optimized for
 - What you deliberately did not optimize for
 - How the system could evolve as scale increases
-

Suggested Deep-Dive Areas (Optional)

You may choose to explore one of these areas in detail:

Provenance Data Structures

- Merkle DAGs
- Hash chains
- Transparency logs
- Anchoring to public ledgers

Agent Identity Systems

- Key hierarchies
- Delegation models
- Revocation mechanisms

Workflow Engines

- Saga patterns
- Deterministic replay
- Checkpointing strategies

Audit Systems

- Graph-based event reconstruction
- Query indexing
- Proof generation

The Determinism Boundary

- How to wrap non-deterministic AI outputs in deterministic guarantees

What We Evaluate

We are not looking for a perfect answer.

We care about:

- System design clarity
 - Reasoning about tradeoffs
 - Security awareness
 - Distributed systems intuition
 - Ability to reason about AI systems in production
-

Submission Format

Please submit:

- A written document (preferred), or
- Slides / architecture diagrams if you prefer

Typical responses are **3–6 pages**.

We recommend spending **2–4 hours** on this exercise.

Follow-Up Discussion

After submission, we will schedule a conversation to walk through:

- your design decisions
- tradeoffs you considered
- how you might evolve the system over time